Zen: A Foundation Language Model for Instruction Following

Technical Report v2025.01

Zen LM Research Team
research@zenlm.org

January 2025

Abstract

We present **Zen**, a 7 billion parameter instruction-following language model serving as the foundation of the Zen family of models. Trained on 3 trillion tokens of high-quality multilingual data using the Zen MoDE (Mixture of Distilled Experts) architecture, Zen achieves competitive performance on standard natural language understanding, reasoning, and code generation benchmarks while maintaining efficient inference characteristics suitable for broad deployment. Post-training combines supervised fine-tuning (SFT) with reinforcement learning from human feedback (RLHF), yielding a model that reliably follows instructions across 100+ languages with a 32K token context window. Zen establishes a strong general-purpose baseline that downstream specialized models in the Zen family extend and refine.

Contents

1 Introduction **3**

2 Architecture **3**
2.1 Overview . . . 3
2.2 Grouped Query Attention . . . 4
2.3 Rotary Position Embeddings . . . 4
2.4 Feed-Forward Network . . . 4
2.5 Tokenization . . . 4

3 Training Methodology **4**
3.1 Pretraining Data . . . 4
3.2 Pretraining Procedure . . . 5
3.3 Supervised Fine-Tuning . . . 5
3.4 Reinforcement Learning from Human Feedback . . . 5

4 Evaluation **6**
4.1 Benchmark Results . . . 6
4.2 Long-Context Evaluation . . . 6
4.3 Multilingual Evaluation . . . 6
4.4 Safety and Alignment . . . 6

5 Inference Efficiency **7**

6 Related Work **7**

7 Limitations **7**

8 Conclusion **7**

1 Introduction

Foundation language models trained at scale on diverse internet-scale corpora have demonstrated remarkable generalization across tasks without task-specific fine-tuning [1, 2]. The key challenge in deploying such models for real-world applications is aligning raw language modeling capability with human intent: models must not only predict likely continuations but follow instructions accurately, refuse harmful requests, and remain calibrated about uncertainty.

Zen is our answer to this challenge at the 7B scale. We make the following contributions:

- A 7B parameter model trained on 3T tokens with a carefully curated multilingual corpus covering 100+ languages, achieving strong cross-lingual transfer.
- A post-training pipeline combining multi-turn SFT on instruction-response pairs with RLHF using a separately trained reward model, improving instruction adherence and reducing harmful outputs.
- A 32K token context window via rotary position embeddings (RoPE) with extended base frequency, enabling long-document summarization and multi-turn dialogue without positional degradation.
- Strong benchmark results competitive with models of comparable and larger scale, while maintaining fast inference throughput suitable for latency-sensitive applications.

Zen occupies the entry tier of the Zen family. Larger and specialized models (Zen-Pro at 72B, Zen-Max at 480B MoE, Zen-VL for vision, Zen-Code for code) build on the same infrastructure and training methodology introduced here.

2 Architecture

2.1 Overview

Zen is a dense decoder-only transformer following the Zen MoDE architecture. Table 1 summarizes the key hyperparameters.

Table 1: Zen architecture hyperparameters.

Hyperparameter	**Value**
Parameters (total)	7.2B
Layers	32
Attention heads	32
KV heads (GQA)	8
Hidden dimension	4096
FFN intermediate dimension	11008
Vocabulary size	151,936
Context length (training)	32,768
Position encoding	RoPE ($\theta = 1{,}000{,}000$)
Activation function	SiLU
Normalization	RMSNorm
Tied embeddings	No

2.2 Grouped Query Attention

Zen employs Grouped Query Attention (GQA) [3] with 32 query heads and 8 key-value heads. This reduces the KV cache memory footprint by 4× relative to multi-head attention at equivalent query capacity, enabling larger effective batch sizes during inference. Attention is computed as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V \tag{1}$$

where $Q \in \mathbb{R}^{n \times d_k}$, $K, V \in \mathbb{R}^{m \times d_k}$, and each group of query heads shares a single key and value projection.

2.3 Rotary Position Embeddings

Rotary position embeddings (RoPE) [4] encode position as a rotation in the complex plane applied to query and key vectors:

$$\tilde{q}_m = q_m e^{im\theta}, \quad \tilde{k}_n = k_n e^{in\theta} \tag{2}$$

We extend the base frequency to $\theta = 1{,}000{,}000$ (from the standard 10,000), allowing the model to generalize to 32K tokens with minimal perplexity degradation at long contexts.

2.4 Feed-Forward Network

Each transformer layer contains a SwiGLU [5] feed-forward block:

$$\text{FFN}(x) = (\text{SiLU}(xW_{\text{gate}}) \odot xW_{\text{up}})\, W_{\text{down}} \tag{3}$$

The intermediate dimension of 11,008 is chosen to be a multiple of 64 for hardware alignment while maintaining the standard expansion ratio of approximately 2.67× the hidden dimension.

2.5 Tokenization

We use a byte-pair encoding (BPE) tokenizer with a vocabulary of 151,936 tokens. The tokenizer was trained on a 50B token sample of the pretraining corpus to ensure adequate coverage of code, mathematical notation, and scripts across all 100+ supported languages. Special tokens include system, user, and assistant turn markers for instruction-tuned inference.

3 Training Methodology

3.1 Pretraining Data

The 3 trillion token pretraining corpus is assembled from the following domains:

Table 2: Pretraining data composition.

Domain	**Tokens (B)**	**Fraction**
Web text (filtered)	1800	60.0%
Books and long-form	450	15.0%
Code	300	10.0%
Scientific articles	150	5.0%
Multilingual web	240	8.0%
Math and STEM	60	2.0%
Total	3000	100.0%

Data quality filtering applies a sequence of heuristics: language identification, deduplication via MinHash [6], perplexity filtering against a small n-gram model, and classifier-based toxicity and quality scoring. Mathematical and code data undergo additional correctness filtering where possible (e.g., code compilation checks).

3.2 Pretraining Procedure

Pretraining uses the AdamW optimizer [7] with:

- Learning rate: 3×10^{-4} with cosine decay to 3×10^{-5}
- Warm-up: 2000 steps
- Weight decay: 0.1
- Gradient clipping: 1.0
- Batch size: 4M tokens (dynamic packing, no padding)
- Precision: BF16 mixed precision
- Parallelism: tensor parallelism $\times 8$, data parallelism $\times 256$

The training runs for approximately 750K steps on 8192 H100 GPUs, consuming approximately 1.75×10^{23} FLOPs. We apply a two-stage cooldown: the final 5% of training tokens are drawn exclusively from high-quality curated sources to sharpen instruction-following priors.

3.3 Supervised Fine-Tuning

Post-pretraining SFT uses 5 million instruction-response pairs spanning:

- General question answering and open-ended generation
- Multi-turn dialogue with persona consistency
- Code generation and debugging
- Summarization and document analysis
- Mathematical reasoning with step-by-step solutions
- Multilingual instruction pairs (40 high-resource languages)

SFT trains for 3 epochs with a learning rate of 2×10^{-5}, packing sequences up to 8K tokens per sample. Loss is computed only on assistant turn tokens (response masking).

3.4 Reinforcement Learning from Human Feedback

RLHF applies Proximal Policy Optimization (PPO) [8] against a reward model trained on 800K human preference comparisons. The reward model shares the same architecture but is trained with a scalar reward head. RLHF training uses:

$$\mathcal{L}_{\text{PPO}} = \mathbb{E}\Big[\min\Big(r_t(\theta)\hat{A}_t,\ \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\Big)\Big] - \beta D_{\text{KL}}(\pi_\theta \| \pi_{\text{ref}}) \tag{4}$$

with clipping parameter $\epsilon = 0.2$ and KL penalty coefficient $\beta = 0.05$. The KL penalty prevents excessive drift from the SFT policy while allowing the model to improve on preference dimensions.

4 Evaluation

4.1 Benchmark Results

Table 3 reports Zen's performance on standard evaluation benchmarks compared to representative models at similar parameter counts.

Table 3: Benchmark results. All numbers are zero-shot or few-shot as standard for each benchmark.

Benchmark	Zen (7B)	Competitor A (7B)	Competitor B (8B)	Competitor C (13
MMLU (5-shot)	**72.3**	70.1	68.9	71.8
HellaSwag (0-shot)	**85.4**	83.2	82.7	84.6
ARC-Challenge (0-shot)	59.4	58.2	57.9	60.1
WinoGrande (0-shot)	74.1	73.5	72.4	74.8
GSM8K (8-shot, CoT)	**78.9**	74.3	72.1	76.5
HumanEval (pass@1)	**71.2**	67.4	65.8	69.3
MBPP (pass@1)	64.8	62.1	61.3	65.2
TriviaQA (1-shot)	68.3	66.7	65.4	68.9
NaturalQuestions	31.4	29.8	28.6	31.7

4.2 Long-Context Evaluation

We evaluate long-context capability using the RULER benchmark [9] and needle-in-a-haystack (NIAH) retrieval across context lengths from 1K to 32K tokens.

Table 4: RULER scores at various context lengths (maximum possible: 100).

Model	4K	8K	16K	32K
Zen (7B)	94.1	91.7	88.3	83.2
Competitor A (7B)	93.8	88.4	79.1	61.4
Competitor B (8B)	92.3	87.9	77.6	58.2

Zen maintains strong retrieval accuracy through 32K tokens, reflecting the extended RoPE base frequency and long-context data mixed into pretraining.

4.3 Multilingual Evaluation

Table 5: Multilingual MMLU accuracy (5-shot) on selected language tracks.

Model	EN	ZH	DE	FR	AR
Zen (7B)	72.3	70.8	68.4	69.1	63.7
Competitor A (7B)	70.1	61.2	62.3	63.7	54.1

4.4 Safety and Alignment

We evaluate instruction-following fidelity using IFEval [10] (prompt-level accuracy 72.8%, instruction-level accuracy 80.3%) and safety alignment using TruthfulQA [11] (truthful: 58.4%, truthful+informative: 47.2%). Refusal rates on harmful prompts from the AdvBench dataset reach 94.3%, indicating strong alignment with safety guidelines.

5 Inference Efficiency

Table 6: Inference throughput and latency on a single A100-80GB GPU.

Metric	BF16	INT4 (GPTQ)
Throughput (tok/s, batch=1)	87	156
Throughput (tok/s, batch=32)	2,840	4,910
TTFT P50 (ms, 1K prompt)	42	24
Memory (GB)	14.8	4.6

The 7B scale allows deployment on a single consumer GPU (RTX 4090 / 24GB VRAM) in BF16, or on commodity hardware in 4-bit quantization, making Zen widely accessible for on-premise and edge deployments.

6 Related Work

Dense decoder-only transformers have been the dominant paradigm since GPT-3 [1]. At the 7B scale, models such as LLaMA [12] and its successors demonstrated that careful data curation and training at scale yield strong transfer. Instruction tuning via SFT [13] and preference optimization via RLHF [14] and DPO [15] have become standard post-training steps. Zen's architecture choices (GQA, SwiGLU, RoPE with extended base frequency) follow a well-validated pattern while the training corpus and post-training data pipeline reflect our own curation methodology.

Long-context capability has been addressed through YaRN [16] and similar RoPE scaling techniques; we adopt an extended base frequency approach as a simpler, hardware-friendly alternative. Multilingual capability at the 7B scale has been studied in BLOOM [17] and similar work; our data weighting strategy achieves comparable cross-lingual transfer while prioritizing high-resource language quality.

7 Limitations

Despite strong benchmark performance, Zen inherits known limitations of autoregressive language models. The model can hallucinate plausible-sounding but incorrect facts, particularly for low-frequency knowledge. Mathematical reasoning degrades on problems requiring more than $\sim$8 chain-of-thought steps. The 32K context, while substantial, may be insufficient for full-document legal or scientific corpora. Bias and toxicity reduction through RLHF is incomplete; adversarial prompt engineering can elicit policy-violating outputs at non-trivial rates.

8 Conclusion

Zen establishes a competitive 7B foundation model for the Zen family, combining a clean Zen MoDE architecture with a large, carefully curated pretraining corpus and a rigorous post-training alignment pipeline. Benchmark results demonstrate that Zen is competitive with or superior to models of comparable scale across language understanding, mathematical reasoning, and code generation tasks. The model's efficient inference profile enables broad deployment across hardware tiers, from cloud inference to on-device applications. Future work will focus on improving mathematical reasoning depth, reducing hallucination rates, and extending context to 128K tokens in forthcoming Zen family releases.

References

[1] T. Brown et al., "Language Models are Few-Shot Learners," *NeurIPS*, 2020.

[2] J. Wei et al., "Emergent Abilities of Large Language Models," *TMLR*, 2022.

[3] J. Ainslie et al., "GQA: Training Generalized Multi-Query Transformer Models," *EMNLP*, 2023.

[4] J. Su et al., "RoFormer: Enhanced Transformer with Rotary Position Embedding," *arXiv:2104.09864*, 2021.

[5] N. Shazeer, "GLU Variants Improve Transformer," *arXiv:2002.05202*, 2020.

[6] A. Broder, "On the resemblance and containment of documents," *Compression and Complexity of Sequences*, 1997.

[7] I. Loshchilov and F. Hutter, "Decoupled Weight Decay Regularization," *ICLR*, 2019.

[8] J. Schulman et al., "Proximal Policy Optimization Algorithms," *arXiv:1707.06347*, 2017.

[9] C.-Y. Hsieh et al., "RULER: What's the Real Context Size of Your Long-Context Language Models?" *arXiv:2404.06654*, 2024.

[10] J. Zhou et al., "Instruction-Following Evaluation for Large Language Models," *arXiv:2311.07911*, 2023.

[11] S. Lin, J. Hilton, and O. Evans, "TruthfulQA: Measuring How Models Mimic Human Falsehoods," *ACL*, 2022.

[12] H. Touvron et al., "LLaMA: Open and Efficient Foundation Language Models," *arXiv:2302.13971*, 2023.

[13] J. Wei et al., "Finetuned Language Models Are Zero-Shot Learners," *ICLR*, 2022.

[14] L. Ouyang et al., "Training Language Models to Follow Instructions with Human Feedback," *NeurIPS*, 2022.

[15] R. Rafailov et al., "Direct Preference Optimization: Your Language Model is Secretly a Reward Model," *NeurIPS*, 2023.

[16] B. Peng et al., "YaRN: Efficient Context Window Extension of Large Language Models," *arXiv:2309.00071*, 2023.

[17] T. Scao et al., "BLOOM: A 176B-Parameter Open-Access Multilingual Language Model," *arXiv:2211.05100*, 2022.